

Product Inventory CRUD

Week 10 — Day 2 • Guided Lab

GUIDED LAB

English copy for submission — clean documentation of the exact guided lab.

Lab outcome

A fresh Django project using the Product model to practice Create, Read, Update, and Delete operations through the Django ORM.

Lab checklist

01	Create three products
02	Retrieve every product and display its primary key
03	Filter active products with stock above zero
04	Find products whose name contains a search term
05	Retrieve one product by PK and handle a missing PK
06	Update one product with <code>save(update_fields=...)</code>
07	Deactivate products with <code>QuerySet.update()</code>
08	Delete only inactive products with zero stock

Project setup

The lab was created from scratch in a new project folder.

```
python -m venv venv
venv\Scripts\activate
pip install django
django-admin startproject product_crud .
python manage.py startapp products
python manage.py check
python manage.py makemigrations products
python manage.py migrate
```

Product model

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)
```

01 — Create three products

Create

```
product1 = Product.objects.create(
    name="Keyboard",
    price=299.00,
    stock=10
)

product2 = Product.objects.create(
    name="Mouse",
    price=99.00,
    stock=25
)

product3 = Product.objects.create(
    name="Monitor",
    price=799.00,
    stock=5
)
```

What happened

Creates three Product rows immediately. Each create() call saves the row and returns the created Product instance.

02 — Retrieve every product and display its primary key

Read

```
products = Product.objects.all()

for product in products:
    print(product.pk, product.name)
```

What happened

all() returns a QuerySet containing all Product records. Iterating over it retrieves the rows. pk is the primary key of each row.

03 — Filter active products with stock above zero

[Read](#)

```
active_products = Product.objects.filter(
    is_active=True,
    stock__gt=0
)

for product in active_products:
    print(product.name, product.stock)
```

What happened filter() selects rows matching both conditions. stock__gt=0 means stock is greater than zero.

04 — Find products whose name contains a search term

[Read](#)

```
search_results = Product.objects.filter(
    name__icontains="key"
)

for product in search_results:
    print(product.name)
```

What happened icontains searches anywhere inside the name without caring about letter case. The result here was Keyboard.

05 — Retrieve one product by PK and handle a missing PK

Read

```
product = Product.objects.get(pk=1)
print(product.name)

try:
    product = Product.objects.get(pk=999)
except Product.DoesNotExist:
    product = None

print(product)
```

What happened `get()` is used when exactly one object is expected. If no matching row exists, Django raises `DoesNotExist`, so `try/except` handles the missing record safely.

06 — Update one product with `save(update_fields=...)`

Update

```
product = Product.objects.get(pk=1)

product.price = 279.00
product.stock = 8

product.save(update_fields=["price", "stock"])

product = Product.objects.get(pk=1)
print(product.name, product.price, product.stock)
```

What happened We loaded one `Product` instance, changed price and stock, then saved those fields. Verification printed: Keyboard 279.00 8.

07 — Deactivate products with QuerySet.update()

Update

```
changed = Product.objects.filter(
    stock__gt=0
).update(is_active=False)

print(changed)
```

What happened

QuerySet.update() changes every matching row directly in the database and returns the number of affected rows. In this lab it returned 3.

08 — Delete only inactive products with zero stock

Delete

```
Product.objects.filter(pk=2).update(stock=0)

deleted = Product.objects.filter(
    is_active=False,
    stock=0
).delete()

print(deleted)
```

What happened

First, Mouse (pk=2) was changed to stock=0. Then the delete filter matched only inactive rows with zero stock. The result showed one Product row deleted.

Exit Ticket

1. `get()`

Retrieve one object; missing rows raise `DoesNotExist`.

2. `filter()`

Build a `QuerySet` containing matching rows.



3. `save()`

Save changes made to a model instance.



4. `update()`

Bulk-update matching rows and return the affected-row count.



5. `delete()`

Delete matching rows and return deletion counts.



Final takeaway

CRUD = Create + Read + Update + Delete. Django ORM lets you perform these database operations using Python objects and `QuerySets`.

